ESPECIFICAÇÃO TÉCNICA E OPERACIONAL V2.0

Documentação Completa do Projeto: CyberGuard Bot

Compêndio exaustivo de todas as regras de negócio, expressões regulares, validações, máquinas de estado, esquemas SQL, permissões, agendadores (crons) e integrações de todos os módulos do sistema.

1. Visão Geral, Arquitetura e Ciclo de Inicialização

O CyberGuard Bot é a plataforma central de governança, conformidade jurídica e automação operacional do projeto de extensão CyberGuard (UTFPR - Câmpus Cornélio Procópio). Desenvolvido em Node.js (v18+) sobre Discord.js v14 e PostgreSQL, o sistema integra geração e validação criptográfica de documentos PDF (ICP-Brasil / Gov.br), ponto eletrônico híbrido (manual e por voz), sincronização em nuvem (Notion API e Google Sheets API), gestão disciplinar com contraditório e transcrição de atas de reunião com Inteligência Artificial (Google Gemini 2.5 Flash).

1.1. Stack Tecnológica e Módulos Core

Camada / Módulo	Tecnologia / Biblioteca	Responsabilidade no Sistema
Gateway & Voz	discord.js v14, @discordjs/voice, prism-media	Eventos do servidor, Slash Commands, Modais, DMs e captura de streams Opus em chamadas de voz.
Persistência Relacional	PostgreSQL (pg Pool)	12 tabelas relacionais com índices parciais de integridade, histórico de auditoria e metadados.
Documentos & AcroForm	docxtemplater, pizzip, pdf-lib, pdfjs-dist, LibreOffice	Preenchimento de templates .docx, conversão headless para .pdf, injeção de AcroForm e leitura de assinaturas /Sig.
Mensageria E-mail	nodemailer (SMTP) & imapflow (IMAP)	Envio de termos e declarações para assinatura e leitura autônoma de respostas na INBOX.
Integrações Cloud	Notion API (2025-09-03 / 2022-06-28) & Google Sheets v4	Gestão de tarefas/progresso, base histórica de membros (Ativo/Inativo/Expulso) e planilhas de horas.
Inteligência Artificial	@google/generative-ai (Gemini 2.5 Flash)	Transcrição em lotes de áudio 16kHz Mono e estruturação automática de Atas de Reunião.
Motor Gráfico 2D	@napi-rs/canvas	Renderização server-side do Organograma (2400px), Pódio de Ranking Semanal, Catálogo de Membros e Cartões.

1.2. Intents, Partials e Sequência de Boot (src/index.js)

- Gateway Intents ativos: Guilds, GuildMembers, GuildMessages, DirectMessages, MessageContent e GuildVoiceStates.
- Partials habilitados: Partials.Channel e Partials.Message (necessários para recepção de DMs e anexos PDF sem cache prévio).
- Ordem estrita no evento client.once("ready"):
  - 1) db.initSchema(): cria e migra automaticamente todas as tabelas e constraints do banco principal.
  - 2) Inicialização de Crons: iniciarLembretes (09:00), iniciarCronAdvertencias (06:00) e iniciarCronDeclaracoes (a cada 1 min + 10s pós-boot).
  - 3) ponto.init(client) -> kovLogs.init(client): cria schema de ponto, reconcilia call de voz, inicia tick de 3 min e faz backfill de logs do KoV.
  - 4) Fetch Único da Guilda (guild.members.fetch()): executado uma única vez e compartilhado entre processarMembrosPendentes e processarSaidasPerdidas para evitar GatewayRateLimitError (opcode 8).
  - 5) popularCacheDeMembros(guild) + hierarquia.agendarAtualizacaoHierarquia(guild): paginação via REST (1000 membros/lote) com fallback de até 3 tentativas respeitando retry_after.

1.3. Regra Global de Resolução de Nome Real (db.resolverNomeReal)

Em todos os relatórios, planilhas, Notion, atas e organogramas, o nome completo do integrante é resolvido seguindo a prioridade estrita: (1º) coluna "nome" da tabela "membros" -> (2º) coluna "nome" da tabela "cadastro_legado" -> (3º) fallback para displayName, globalName ou username do Discord.

2. Módulo de Onboarding, Validações Cadastrais e Aprovação do RH

O módulo de Onboarding (src/onboarding.js, src/validators.js, src/approval.js, src/catchUp.js) conduz o ingresso de novos voluntários por DM, valida rigorosamente cada dado informado e submete o perfil à deliberação do RH.

2.1. Gatilhos de Entrada e Proteção de Membros Antigos (catchUp.js)

- Entrada ao Vivo (guildMemberAdd): dispara iniciarOnboarding(member) imediatamente ao entrar no servidor.
- Reconciliação Pós-Boot (processarMembrosPendentes): varre os membros presentes no servidor e inicia o onboarding apenas para quem atender cumulativamente a 3 regras:
  - Regra 1: não ser bot (!membro.user.bot).
  - Regra 2: não possuir NENHUM cargo além de @everyone (membro.roles.cache.size <= 1), protegendo veteranos sem registro no banco.
  - Regra 3: não existir na tabela membros ou estar parado no status inicial "coletando_dados" (ignora status pendente_aprovacao, aguardando_assinatura, revisao_manual, concluido e rejeitado). O disparo ocorre com intervalo de 3.000 ms (3s) por membro.

2.2. As 13 Etapas de Coleta e Regras de Validação (src/validators.js)

Todas as entradas passam por .trim() antes da validação. Cada pergunta ou menu interativo possui timeout individual de 10 minutos (600.000 ms).

Ordem / Campo	Validador / Regex Estrito	Regra de Negócio e Mensagem de Erro
1. nome	validators.naoVazio (trim().length > 0)	Nome completo do voluntário. Não permite string vazia ou apenas espaços.
2. data_nascimento	/^(\d{2})\/(\d{2})\/(\d{4})$/ + Date check	Exige dd/mm/aaaa, mês entre 1 e 12 e verifica se new Date(ano, mes-1, dia) mantém o mesmo dia/mês/ano (bloqueia datas inexistentes como 31/02).
3. nacionalidade	validators.naoVazio	Nacionalidade declarada para o termo jurídico.
4. cpf	/^\d{3}\.\d{3}\.\d{3}-\d{2}$/	Exige máscara estrita com pontos e traço (111.222.333-44).
5. curso	validators.naoVazio	Nome do curso de graduação na UTFPR.
6. periodo	validators.naoVazio	Período/semestre atual (ex.: 5º). Injetado em {periodo} e {período}.
7. ra	/^\d+$/	Registro Acadêmico contendo exclusivamente dígitos numéricos, sem pontos ou letras.
8. endereco	/^(Rua|Av\.?|Avenida)\s+.+,\s*(n[ºo°]\.?\s*)?\d+ [A-Za-z]?\s*-\s*.+$/i	Exige início com Rua/Av/Av./Avenida, logradouro, vírgula, número (com letra opcional, ex.: 123A), hífen e bairro.
9. cidade	validators.naoVazio	Cidade de residência atual.
10. estado	validators.naoVazio	Unidade Federativa (UF).
11. telefone	validators.naoVazio	Telefone de contato com DDD.
12. email	validators.naoVazio	E-mail do integrante (usado para convite no Notion e registros).
13. area_interesse	StringSelectMenu (customId: area_interesse)	Opções padrão: Social Media & Copy, Recursos Humanos, Design, Hardware Hacking, Hacking Ético.

2.3. Loop de Confirmação, Retificação e Tratamento de Abandono

- Ao concluir as 13 etapas, o bot exibe um resumo completo com dois botões: "Está tudo certo" (confirmar_dados) e "Quero corrigir algo" (corrigir_dados).
- Clicar em "Quero corrigir algo" abre o menu "campo_corrigir" com os 13 campos; o usuário refaz apenas o campo selecionado (com a mesma validação estrita) e retorna à tela de confirmação.
- Timeout de 10 minutos (Abandono): se o usuário não responder a qualquer etapa em 10 minutos, o bot grava registro na tabela "historico_recusas" com tipo = "abandonado", remove o registro de "membros" (db.removerMembro) e avisa por DM.

2.4. Deliberação do RH: Aprovação vs. Rejeição (src/approval.js)

- Alerta de Histórico Anterior: ao enviar o card para o canal do RH (CHANNEL_RH_ID), o bot consulta "historico_recusas" buscando coincidência por discord_id OU ra OU LOWER(nome). Se encontrar passagens anteriores (rejeições ou abandonos), exibe um bloco de alerta exclusivo no card do RH.
- Permissão Estrita: apenas integrantes com o cargo nominal configurado em ROLE_RH_NOME ("Recursos Humanos") podem clicar em "Aprovar" (aprovar_<id>) ou "Rejeitar" (rejeitar_<id>).
- Trava de Concorrência: exige status === "pendente_aprovacao" e substitui os botões imediatamente por botões desabilitados ("Processando...").
- Fluxo de Rejeição (rejeitar_<id>):
  - 1) Solicita no canal que o membro do RH digite o motivo em até 5 minutos (fallback automático se expirar).
  - 2) Registra em "historico_recusas" (tipo = "rejeitado"), envia DM cordial ao candidato (sem expor o motivo interno), executa guildMember.kick() caso EXPULSAR_AO_REJEITAR !== "false", e apaga o registro de "membros".
- Fluxo de Aprovação (aprovar_<id>):
  - 1) Define status = "aguardando_assinatura", data_aprovacao = now() e data_limite_assinatura = now() + PRAZO_DIAS (padrão 7 dias).
  - 2) Gera o PDF personalizado do Termo de Voluntariado com campo AcroForm preenchível.
  - 3) Grava linha na planilha de cadastro (Google Sheets) e cadastra o membro como "Ativo" na base de Membros do Notion.
  - 4) Publica cartão gráfico de boas-vindas no canal CHANNEL_BOASVINDAS_ID e atribui os cargos no Discord (ROLE_APROVADO_NOME + área escolhida, usando fallback pelo ID 1502772363178147860 para Hacking Ético).
  - 5) Envia lembrete no canal do RH com o e-mail do membro para liberação de acesso como convidado no Workspace do Notion.
  - 6) Envia 3 mensagens na DM do aprovado: (a) Boas-vindas com links do WhatsApp Geral e da Área + Termo em PDF; (b) Tutorial de Assinatura Gov.br em PDF; (c) Manual de Conduta completo com link do Google Forms de free-time.

3. Termo de Voluntariado: Geração, Validação Gov.br e Revisão Manual

O ciclo do Termo de Voluntariado (src/termService.js, src/dataAssinatura.js, src/recepcaoTermo.js, src/reminders.js) garante que todos os membros possuam termo assinado digitalmente com certificado ICP-Brasil / Gov.br.

3.1. Geração Dinâmica do PDF e Injeção AcroForm (src/termService.js)

- Renderização DOCX: injeta os dados cadastrais em templates/termo_voluntariado.docx via docxtemplater (tags ausentes recebem "[[FALTA PREENCHER: <tag>]]", exceto data_assinatura que fica em branco).
- Conversão PDF: executa LibreOffice em modo headless (soffice --headless --convert-to pdf).
- Localização de Coordenadas e AcroForm: varre as páginas com pdfjs-dist procurando o item de texto exato "Data:". Ao localizar, usa pdf-lib para injetar um campo de texto interativo (TextField "data_assinatura") posicionado em x = pos.x + pos.width + 6, y = pos.y - 3, largura 100, altura 16 e borda azul.

3.2. As 6 Regras de Validação Automática 100% Assertiva (src/dataAssinatura.js)

Quando o membro envia um arquivo .pdf na DM do bot enquanto está em status "aguardando_assinatura" ou "revisao_manual", o motor validarTermoDevolvido() avalia simultaneamente assinatura e data (acumulando todos os erros encontrados para informar o aluno de uma só vez):

Etapa	Inspeção Técnica Realizada	Critério de Aprovação / Motivo de Falha
1. Objeto /Sig	Enumera objetos indiretos do PDF buscando Type === "/Sig" (ou SubFilter) com "/ByteRange" e "/Contents".	Exige assinatura criptográfica Gov.br; detecta se o aluno usou "Imprimir > Salvar como PDF" e perdeu o certificado.
2. Widget /Sig	Inspeciona widgets via pdf-lib (Annots e ref /P) com fallback em pdfjs-dist (getAnnotations fieldType "Sig").	Extrai com 100% de precisão a página (pageIndex) e o retângulo [x1, y1, x2, y2] do selo da assinatura.
3. Tabela Aceite	Localiza a seção "ACEITE E CONCORDÂNCIA" e os rótulos na 1ª coluna (x em [50, 120]): "Voluntário(a)" (V maiúsculo) e "Coordenação da ação" / "DIREC".	Elimina falsos positivos com o texto corrido da página 2 ("voluntário(a)" em x=121 e "DIREC" em x=352).
4. Geometria (x,y)	Verifica se o centro (cx, cy) do widget está em x [185, 560] e y [coordenacaoY + 7, voluntarioY + 32].	Distingue exatamente se o aluno assinou na página errada, na linha da Coordenação/DIREC ou fora da tabela.
5. Campo Data:	Lê exclusivamente o campo "Data:" ao lado de "Local: Cornélio Procópio" (via AcroForm data_assinatura, anotação FreeText/Tx ou linha horizontal achatada |y| <= 10).	Nunca usa a data do carimbo Gov.br como fallback; se o aluno deixou o campo "Data:" em branco, acusa o erro com precisão.
6. Janela 90 Dias	Valida calendário real (validarData em dd/mm/aaaa), compara com a data atual no fuso America/Sao_Paulo e limite de 90 dias.	Rejeita formatos inválidos, datas futuras e datas com mais de 90 dias no passado.

3.3. Reprovação Automática Direta na DM e Fallback de Revisão Manual

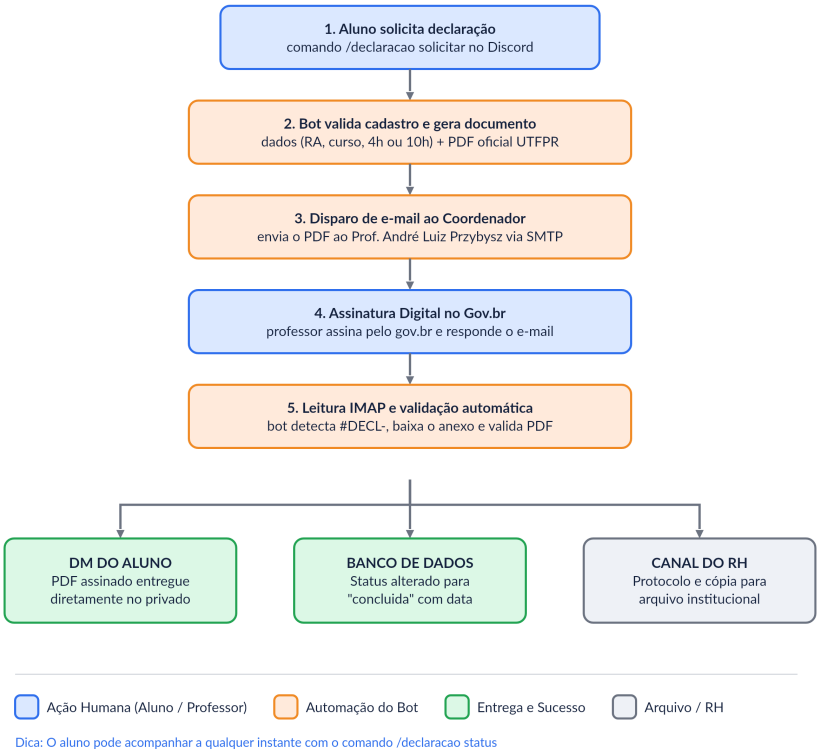
- Reprovação Direta na DM (revisarManualmente = false): sempre que o validador identifica um erro objetivo do aluno (campo "Data:" em branco ou inválido, assinatura fora da linha "Voluntário(a)" ou página errada, ou PDF sem assinatura criptográfica), o bot NÃO sobrecarrega o canal do RH. Ele mantém o membro em "aguardando_assinatura", remove o arquivo temporário e responde imediatamente na DM do aluno listando todos os erros para correção e reenvio.
- Fallback de Revisão Manual pelo RH (revisarManualmente = true): acionado apenas em anomalias técnicas raras (ex.: assinatura criptográfica presente mas sem coordenadas legíveis), publicando card no canal do RH com botões "[OK] Aprovar e Enviar E-mail" (termo_aprovar_<id>) e "[X] Reprovar Termo" (termo_reprovar_<id>).

3.4. Arquivamento Setorial, Envio SMTP e Lembretes Diários

- Arquivamento Permanente: o PDF validado é copiado para docs/Termos-Voluntarios-Assinados/<Subpasta>/termo_assinado_<RA>_<Nome>.pdf, onde <Subpasta> é mapeada automaticamente pela área (HackingEtico, Design, HardwareHacking, Recursos Humanos, SocialMedia, Líderes ou Geral).
- Disparo SMTP (emailService.js): envia o termo assinado para EMAIL_TERMOS_DESTINATARIO (padrão natassyasilva@utfpr.edu.br) com cópia (CC) para EMAIL_TERMOS_CC.
- Cron de Lembretes (reminders.js): roda diariamente às 09:00 (CRON_LEMBRETE = "0 9 * * *") para membros em "aguardando_assinatura" com lembrete_resolvido_manualmente = false. Se dentro do prazo de 7 dias, envia DM com contagem regressiva; se vencido, cobra o membro por DM e alerta o canal do RH com o botão "[OK] Já está tudo certo, parar de avisar" (lembrete_resolvido_<id>).

4. Módulo de Emissão de Declaração de Voluntariado

O módulo de Declarações (commands/declaracao/declaracaoCommand.js, src/declaracaoService.js, src/declaracaoEmail.js) automatiza de ponta a ponta a solicitação, geração de PDF oficial, envio por e-mail ao Coordenador (Prof. André Luiz Przybysz), captura da resposta assinada via IMAP e entrega na DM.

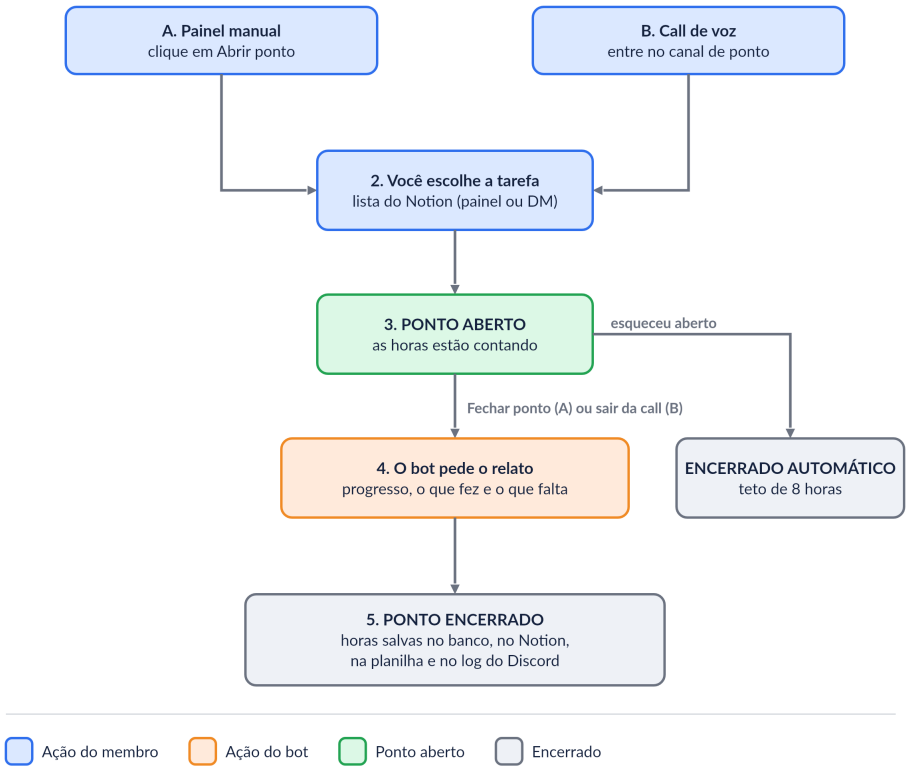


4.1. Regras de Negócio e Validações (/declaracao solicitar e /declaracao status)

- Validação de Cadastro: exige que o solicitante exista na tabela "membros" com "nome" e "ra" preenchidos.
- Bloqueio de Duplicidade Pendente: verifica se o aluno já possui registro na tabela "declaracoes_solicitadas" com status = "aguardando_assinatura_professor". Caso possua, impede nova abertura e exibe o protocolo e a data da solicitação em curso.
- Cálculo Automático de Carga Horária (4 horas vs. 10 horas):
  - 10 horas semanais ("10 horas"): atribuída automaticamente se o solicitante for o Presidente (config.presidenteId) ou possuir o cargo de liderança ("Líderes", "Lideres", ROLE_LIDER_NOME ou PONTO_ROLE_LIDERES_ID).
  - 4 horas semanais ("4 horas"): atribuída automaticamente aos demais membros regulares.
- Data de Início e Protocolo: utiliza membro.data_inicio (com fallback para "05/2026") e gera código único no formato DECL-<RA>-<4 últimos dígitos do timestamp>.
- Geração do PDF Oficial: renderiza templates/declaracao_participacao.docx com a data de solicitação por extenso em português (ex.: "16 de outubro de 2026") e converte via LibreOffice.
- Monitoramento IMAP Autônomo e Sob Demanda (declaracaoEmail.js):
  - Protegido por mutex em memória (isCheckingImap). Roda a cada 1 minuto via cron ("* * * * *"), 10s após o boot e também é forçado em tempo real sempre que um aluno executa /declaracao status.
  - Conecta à INBOX via ImapFlow, busca assuntos contendo "DECL-", extrai o código via regex /#(DECL-[A-Z0-9_-]+)/i, percorre recursivamente a árvore MIME em busca de anexos PDF, valida os 4 primeiros bytes ("%PDF"), salva em docs/Termos-Voluntarios-Assinados/DeclaracoesAssinadas/, marca o e-mail como lido (\Seen), atualiza o banco para "concluida", envia o PDF na DM do aluno e posta cópia no canal do RH.

5. Módulo de Ponto Eletrônico v2, Notion, Sheets, KoV e Ranking

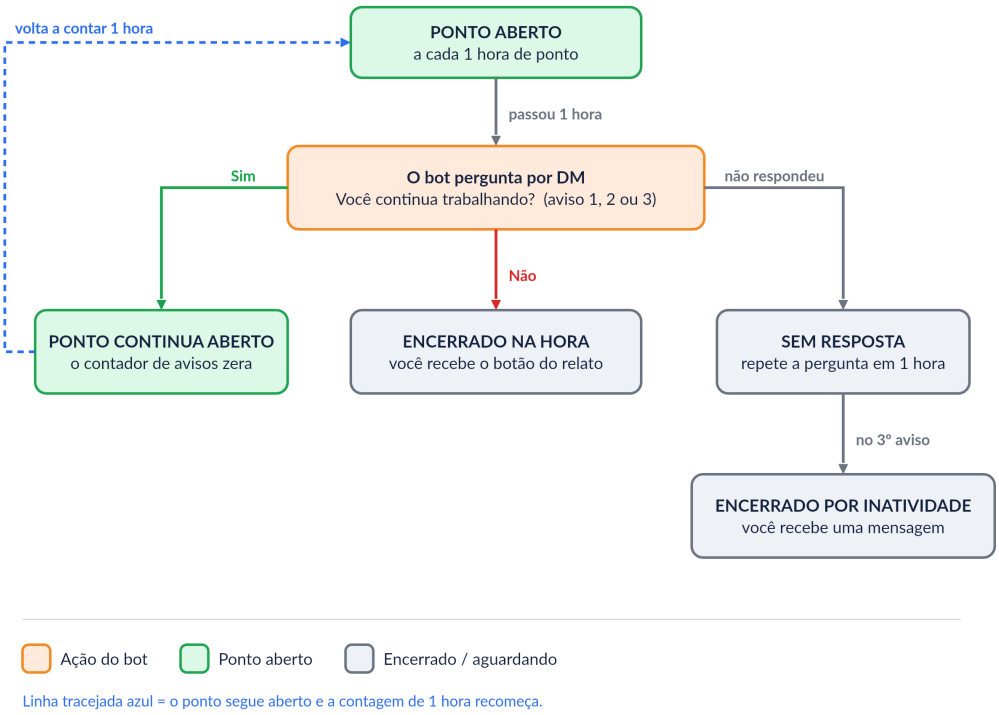
O Ponto Eletrônico v2 (src/ponto.js, src/pontoDb.js, src/pontoSheets.js, src/notion.js, src/notionMembros.js, src/kovLogs.js, src/rankingRender.js) unifica o controle de horas manuais, chamadas de voz e registros legados.



5.1. Regras de Abertura, Criação de Tarefa e Fechamento Manual

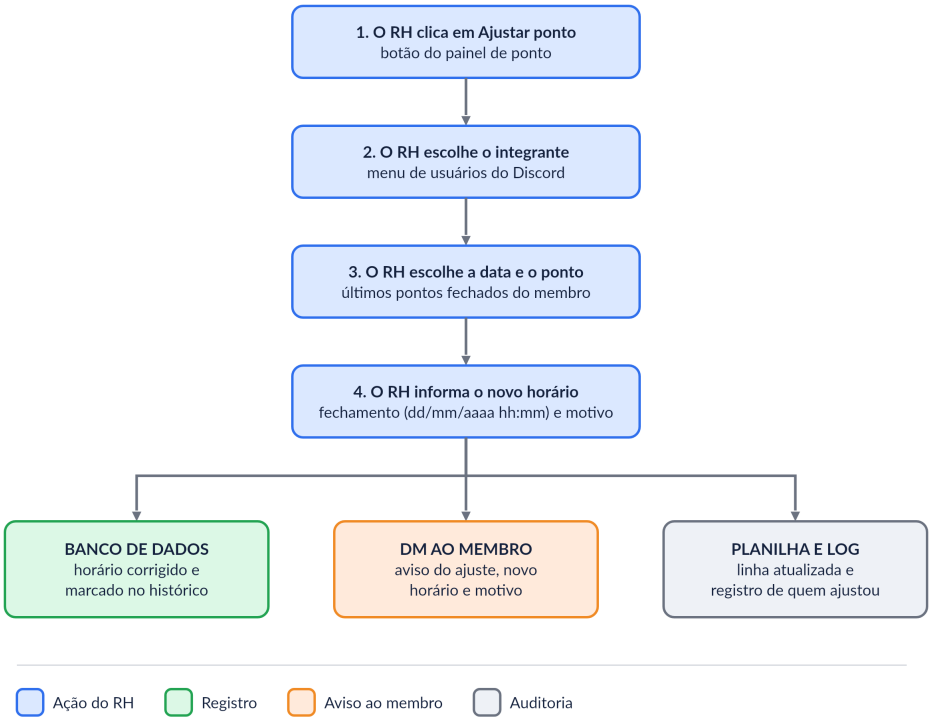
- Unicidade no SGBD: o índice parcial único "pontos_um_aberto_por_usuario" (UNIQUE ON pontos (discord_id) WHERE fechado_em IS NULL) garante no PostgreSQL que nenhum membro tenha dois pontos abertos ao mesmo tempo (código 23505 tratado).
- Listagem e Prioridade de Tarefas do Notion (API 2025-09-03): filtra tarefas com Status != "Concluído" e ordena localmente pela prioridade: (1) Em andamento, (2) A fazer - Essa Semana, (3) Depois de 1 mês, (4) Próximo Semestre, (5) Aguardando Terceiros, (6) Backlogs - Sem Data, (7) Standby Estratégico. Exibe 25 tarefas por página.
- Criação de Nova Tarefa no Notion pelo Discord (ponto:nova_tarefa:<modo>): permite cadastrar uma nova demanda via modal informando Título (obrigatório, até 150 chars) e Progresso Inicial (opcional, 0 a 100, padrão 0). Cria a página no Notion com Status "Em andamento" e Responsável igual ao nome completo resolvido do membro, abrindo ou vinculando o ponto imediatamente.
- Validações Estritas do Modal de Fechamento e Reporte (lerCamposReporte):
  - Campo "progresso": obrigatório, validado por regex /^\d{1,3}$/ (após remover "%") e restrito ao intervalo 0 <= progresso <= 100.
  - Campo "feito" (O que você fez): obrigatório, até 800 caracteres.
  - Campo "falta" (O que falta fazer): até 800 caracteres. É OBRIGATÓRIO sempre que progresso < 100 e opcional apenas quando progresso === 100.
  - Proteção Anti-Perda de Rascunho: se a validação de progresso ou falta falhar, o bot devolve na mensagem efêmera um bloco de código contendo o texto digitado pelo membro para cópia rápida.
- Sincronização com o Notion ao Fechar: se progresso === 100, altera Status da tarefa para "Concluído"; se < 100, mantém/altera para "Em andamento". Atualiza a coluna numérica "Progresso" (criada automaticamente via PATCH no Data Source caso não exista) e publica um comentário detalhado na página da tarefa.

5.2. Ponto por Call de Voz, Checagem Horária de Inatividade e Teto de 8h



- Call de Voz Dedicada (PONTO_VOICE_CHANNEL_ID):
  - Ao conectar: abre ponto com origem = "voz" e tarefa_id = "pendente", enviando DM para escolha ou criação da tarefa.
  - Grace Period de 60 Segundos (GRACE_SAIDA_MS = 60.000 ms): ao sair da call, o bot guarda o timestamp exato da saída (saiuEm) e aguarda 60s. Se o membro reconectar dentro de 60s, o timer é cancelado e a mesma sessão continua aberta.
  - Fechamento de Voz: após 60s fora da call, grava fechado_em = GREATEST(aberto_em, LEAST(saiuEm, now())), reportado = false, e envia DM com botão "Reportar progresso". Pontos abertos manualmente NÃO são fechados ao sair da call.
- Checagem de Inatividade a cada 1 Hora (verificarInatividadePontos):
  - Dispara DM a cada 1 hora de ponto aberto com botões "Sim, continuo trabalhando" (ponto:check_sim:<id>, que zera avisos_inatividade e inclui botão de atalho para fechar ponto) e "Não, já encerrei" (ponto:check_nao:<id>, que abre imediatamente o modal de reporte ou seleção de tarefa pendente).
  - Se o membro ignorar 3 avisos consecutivos e completar mais 1 hora sem resposta (avisos_inatividade >= 3), o ponto é encerrado automaticamente por inatividade (auto_fechado = true, reportado = false).
- Teto Máximo de 8 Horas (PONTO_MAX_HORAS = 8): pontos esquecidos abertos por mais de 8h são encerrados no tick com fechado_em = aberto_em + 8h e auto_fechado = true.
- Lembrete de Reporte Pendente (LEMBRETE_REPORTE_HORAS = 3): pontos fechados por voz/inatividade com reportado = false há mais de 3h recebem um lembrete único em DM (lembrete_enviado = true).

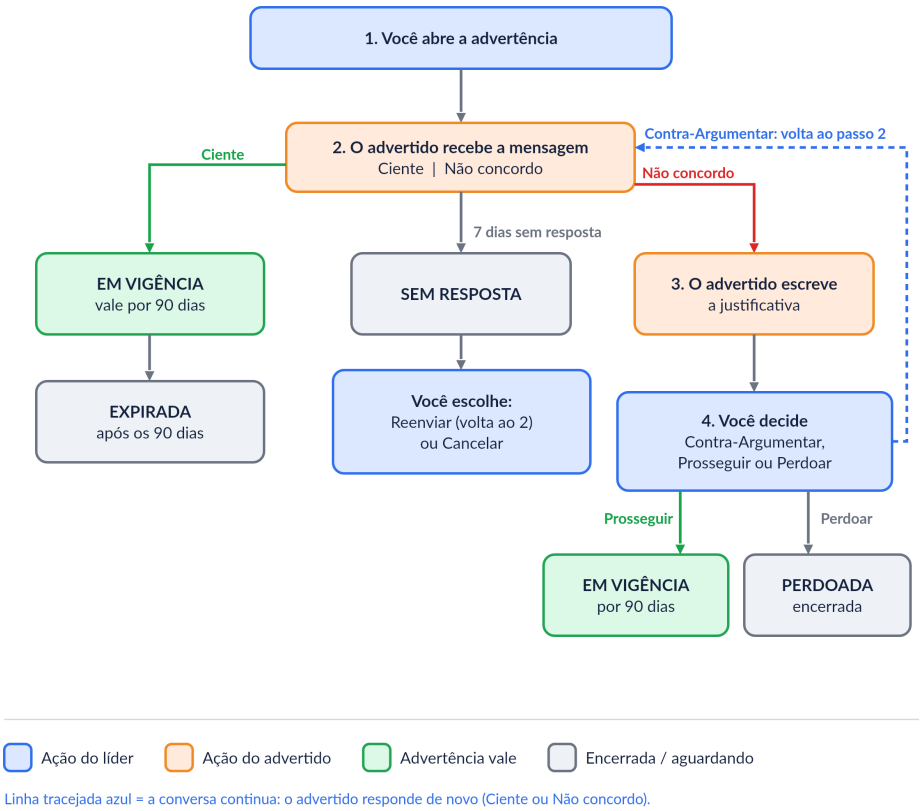
5.3. Gestão do RH (Ajuste de Ponto e Importação) e Sincronização de Planilhas



- Regras e Validações de Ajuste de Ponto pelo RH (/ajustar-ponto e botão Ajustar ponto):
  - Restrito ao cargo PONTO_ROLE_RH_ID. Permite selecionar qualquer ponto já fechado (origem <> "importado").
  - Valida formato dd/mm/aaaa hh:mm via regex /^(\d{1,2})\/(\d{1,2})\/(\d{4})\s+(\d{1,2}):(\d{2})$/ e aplica 3 travas SQL: (1) novo_fechamento > aberto_em; (2) novo_fechamento <= now(); (3) novo_fechamento <= aberto_em + interval "24 hours" (duração máxima de 24h).
  - Preserva o primeiro fechamento em fechado_em_original = COALESCE(fechado_em_original, fechado_em), grava ajustado_por, ajustado_em e ajuste_motivo, sincroniza o Google Sheets e envia DM automática ao colaborador.
- Importação de Horas Legadas (/importar-horas): restrito ao RH. Converte dias (0..3650), horas (0..23), minutos (0..59) e segundos (0..59) em segundos totais (> 0). Se o membro já possuir registro com origem = "importado", substitui o saldo anterior de forma idempotente; caso contrário, cria um novo. Contabiliza no total geral, mas é excluído do ranking semanal.
- Arquitetura de 3 Abas no Google Sheets (src/pontoSheets.js):
  - 1) Aba "Raw Logs do Bot": escrita serializada em fila de Promises; faz upsert pela coluna id_ponto ("p<id>"), remove automaticamente a coluna legada "pontos_produtividade", higieniza rodapés inválidos e insere logs do KoV no topo (A2).
  - 2) Aba "Painel Transparente" (Geral): consolida a soma total de horas de todas as origens (Bot + KoV + Importado) ordenada decrescentemente por horas.
  - 3) Aba "Painel Transparente (Bot)": soma exclusivamente sessões com origem "manual" ou "voz" registradas no CyberGuard Bot.
- Ranking Semanal e Virada de Semana: inicia na segunda-feira 00:00 (America/Sao_Paulo), corta sessões que atravessam a meia-noite de domingo/segunda exatamente nos limites da semana, atualiza a mensagem fixa ao vivo a cada 3 minutos e publica o pódio fechado da semana anterior na virada de semana.

6. Sistema Disciplinar, Fila de Advertências e Contraditório

O módulo disciplinar (src/advertencias.js, src/advertenciasRender.js e comandos /advertir, /painel-advertencias, /advertencias, /minhas-advertencias) assegura o devido processo de contraditório, controle de reincidência e fila sequencial.



6.1. Permissões, Categorias e Escalonamento por Reincidência

- Controle de Acesso: somente usuários listados em AUTORES_ADVERTENCIA_IDS podem abrir advertências, gerenciar a fila, publicar o painel administrativo ou deliberar sobre recursos.
- Categorias Regimentais: Leve (ordem 1), Média (ordem 2) e Grave (ordem 3).
- Validação Estrita de Reincidência: ao marcar reincidência = Sim, o bot verifica se o membro possui advertência anterior na MESMA categoria efetiva com status em ("em_vigencia", "aguardando_aceitacao", "em_analise", "sem_resposta", "na_fila"). Caso não exista, bloqueia a abertura.
- Agravamento Automático: se a reincidência for válida, vincula advertencia_referencia_id e eleva a categoria efetiva em +1 nível: Leve -> Média; Média -> Grave; Grave -> Grave com abertura imediata de votação de expulsão no canal (advertencia_expulsar_sim_<id> / advertencia_expulsar_nao_<id>).

6.2. Fila Automática de Advertências (status = "na_fila") e Revalidação

- Enfileiramento: se o membro já possuir uma advertência pendente de resolução ("aguardando_aceitacao", "em_analise" ou "sem_resposta"), qualquer nova advertência aberta para ele entra automaticamente com status = "na_fila" e data_aguardando_desde = NULL.
- Visibilidade: itens "na_fila" ficam ocultos em /minhas-advertencias até que sejam efetivamente disparados ao membro, podendo ser gerenciados pelo líder no botão "[Aguardando] Fila de Advertências" (Enviar Agora ou Cancelar da Fila).

- Disparo e Revalidação Dinâmica (processarFilaAdvertencias -> ativarAdvertenciaDaFila): assim que a advertência ativa é concluída (Ciente, Prosseguir, Perdoar ou Cancelar), a próxima da fila é disparada automaticamente. Antes do envio, se ela era reincidente, o bot revalida se ainda existe advertência estritamente "em_vigencia" naquela categoria; caso a anterior tenha sido perdoada ou cancelada, a reincidência é desfeita automaticamente (voltando à categoria declarada original).

6.3. Prazos de Contraditório, Rodadas de Argumentos e Cron Diário

- Entrega Resiliente: tenta envio na DM do advertido; se a DM estiver bloqueada, publica no CANAL_ADVERTENCIAS_ID mencionando o membro.
- Botão "Ciente": muda status para "em_vigencia", grava data_resposta e define data_expiracao = now() + 90 dias (DIAS_PARA_EXPIRAR = 90).
- Botão "Não concordo": abre modal de réplica (até 1.500 chars), grava em advertencias_argumentos (tipo = "replica"), muda status para "em_analise" e notifica o autor com 3 opções: (1) Contra-Argumentar (modal de até 1.500 chars que devolve ao membro em "aguardando_aceitacao" e REINICIA o prazo data_aguardando_desde = now() por mais 7 dias); (2) Prosseguir (entra em "em_vigencia" por 90 dias); (3) Perdoar (encerra como "perdoada").
- Cron Diário às 06:00 ("0 6 * * *"): (a) muda para "sem_resposta" advertências em "aguardando_aceitacao" cujo COALESCE(data_aguardando_desde, data_abertura) ultrapassou 7 dias (notificando o autor com botões Reenviar e Cancelar); (b) muda para "expirada" advertências em "em_vigencia" com data_expiracao < now().

7. Módulo de Atas de Reunião com Inteligência Artificial (Gemini)

O módulo de Reuniões (commands/reuniao/*) grava chamadas de voz por falante individual, realiza downsampling de áudio, transcreve em lotes via Google Gemini 2.5 Flash e gera atas formais em PDF.

7.1. Comandos, Captura Opus por Falante e Filtro de Ruído (recorder.js)

- Subcomandos: /reuniao iniciar [titulo] (permite 1 gravação ativa por servidor via Map gravacoesAtivas), /reuniao finalizar, /reuniao historico [quantidade 1..25, padrão 10] e /reuniao reprocessar <id>.
- Captura Individual: escuta connection.receiver.speaking.on("start", userId), subscrevendo ao stream Opus do usuário com EndBehaviorType.AfterSilence (duration: 1.000 ms - corta após 1s de silêncio).
- Decodificação e Filtro Mínimo: decodifica Opus para PCM 48kHz Estéreo 16-bit. Descarta automaticamente arquivos menores que 57.600 bytes (~0,3 segundos de áudio) para eliminar cliques e ruídos curtos.
- Grace Period no Fechamento: ao executar /reuniao finalizar, aguarda até 5.000 ms (em ciclos de 200ms) para que falas em andamento terminem antes de fechar os arquivos WAV.

7.2. Downsampling 16kHz Mono, Batching, Retries e Retenção de Backup

- Downsampling (audioBatch.js): converte cada WAV de 48kHz Estéreo 16-bit para 16kHz Mono 16-bit (fator de decimação 3 com média aritmética dos canais L/R e clamp em [-32768, 32767]).
- Agrupamento em Lotes: divide as falas visando LOTES_ALVO = 12 requisições (mínimo de 3 áudios por lote), concatenando os WAVs separados por 500 ms de silêncio (16.000 bytes de zeros) e aguardando 4.000 ms entre lotes.
- Política de Retry e Cota Diária (gemini.js): realiza até 5 tentativas com backoff exponencial (4000 * 2^tentativa + jitter de até 1000ms) para erros 429 (Rate Limit por minuto) ou 503. Se detectar erro de Cota Diária Esgotada ("PerDay" ou "GenerateRequestsPerDayPerProjectPerModel"), aborta os retries imediatamente e marca os trechos restantes.
- Geração do PDF e Retenção de Áudio (pdfGenerator.js & limpezaBackup.js): gera o PDF da ata estruturada (Resumo, Decisões, Itens de ação com checkboxes e Pontos discutidos). Se 100% dos lotes forem transcritos com sucesso, apaga a pasta de backup imediatamente; se houver falha parcial/cota, preserva a pasta e o manifest.json por 7 dias (permitindo /reuniao reprocessar <id>), com faxina automática diária às 04:00 ("0 4 * * *").

8. Hierarquia Visual, Cartões de Servidor e Governança de Saídas

8.1. Organograma Dinâmico Renderizado (src/hierarquia.js & hierarquiaRender.js)

- Estrutura Visual (2400px de largura): renderiza o card do Presidente no topo, conectado ortogonalmente a 5 times distribuídos em 2 linhas: Linha 1 (Social Media & Copy, Recursos Humanos, Design) e Linha 2 (Hardware Hacking e Hacking Ético). As linhas conectoras da Linha 2 descem pelos vãos laterais (xGap1 e xGap2) sem cruzar sobre o card de Recursos Humanos.

- Unificação Hacking Ético: agrupa automaticamente membros com o cargo Hacking Ético (ID 1502772363178147860 ou nome), Red Team ou Blue Team.
- Destaque de Liderança e Debounce: identifica líderes pelo cargo "Líderes" ou mapa fixo LIDERES_TIME, posicionando-os no topo do card com estrela dourada (*). Aplica debounce de 4.000 ms em mudanças de cargos relevantes e edita a mesma mensagem fixa no canal HIERARQUIA_CHANNEL_ID (salva em bot_state).

8.2. Cartões de Entrada/Saída e Sincronização de Desligamento no Notion

- Cartões Ilustrados (cartaoServidor.js): gera banners 1000x320px em PNG ("EMBARCOU NO SERVIDOR" em azul com mascote sorrindo na aprovação do onboarding; "DESEMBARCOU DO SERVIDOR" em âmbar com mascote triste na saída).
- Detecção de Expulsão via Audit Log e Revogação Notion (guildMemberRemove): quando um membro sai do servidor, o bot inspeciona os últimos 5 registros do Audit Log (actions 20 Kick e 22 Ban nos últimos 30s). Atualiza o cadastro do membro no Notion (notionMembros.js) para Status = "Expulso" (se kick/ban) ou "Inativo" (saída voluntária) com a Data de Saída, e envia alerta no canal do RH informando o e-mail para remoção do acesso de convidado no Workspace do Notion.

9. Referência Consolidada: Banco de Dados, Crons e Variáveis (.env)

9.1. Esquema Relacional PostgreSQL (12 Tabelas)

Tabela SQL	Chave / Índices Únicos	Finalidade e Colunas Principais
membros	PK id, UNIQUE discord_id	Cadastro oficial, status do onboarding/termo, RA, CPF, curso, email, data_inicio, termo_arquivo_path e flags de lembrete/saída.
historico_recusas	PK id, IDX discord_id, ra, nome	Memória permanente de candidatos reprovados pelo RH ("rejeitado") ou que sofreram timeout/saída ("abandonado").
cadastro_legado	PK discord_id	Mapeamento de nomes completos de integrantes antigos pré-onboarding.
bot_state	PK chave	Estado persistente geral (ex.: ID da mensagem fixa da hierarquia).
declaracoes_solicitadas	PK id, UNIQUE codigo	Protocolos #DECL-, carga_horaria (4h/10h), status, caminhos dos PDFs gerado/assinado e message_id_email.
pontos	PK id, UNIQUE aberto/user e kov_msg_id	Sessões de ponto (manual, voz, kov, importado), progresso, feito, falta, avisos_inatividade e auditoria de ajustes do RH.
ponto_meta	PK chave	Metadados do ponto (ranking_ao_vivo_msg e ultima_semana_ranking).
advertencias_categorias	PK id, UNIQUE nome, ordem	Catálogo regimental: Leve (1), Média (2) e Grave (3).
advertencias	PK id, IDX discord_id, status	Ocorrências disciplinares, reincidência, fila ("na_fila"), prazos de 7 e 90 dias e decisão de expulsão.
advertencias_historico_status	PK id, FK advertencia_id	Trilha de auditoria imutável de todas as mudanças de status disciplinar.
advertencias_argumentos	PK id, FK advertencia_id	Histórico cronológico de todas as rodadas de réplicas e contra-argumentos.
atas_reuniao	PK id, IDX guild_id, finalizado_em	Atas geradas pelo Gemini AI, transcrição, participantes JSONB, caminho do PDF e controle de backup de áudio (7 dias).

9.2. Mapa de Todos os Agendadores (Crons) e Loops Temporizados

Processo / Serviço	Frequência / Expressão	Ação Executada Automaticamente
Monitoramento IMAP Declarações	* * * * * (a cada 1 min) + 10s boot	Busca respostas de e-mail com #DECL-, baixa o PDF assinado pelo coordenador, arquiva, entrega na DM e avisa o RH.
Tick do Ponto Eletrônico	A cada 3 min (setInterval) + 20s boot	Envia checks de 1h de inatividade, encerra por inatividade (3 avisos) ou teto (8h), cobra reportes (3h) e atualiza o Ranking ao vivo.
Sync Retry KoV -> Sheets	A cada 15 min (setInterval) + 5s debounce	Reenvia para o Google Sheets eventuais sessões importadas do KoV com sheet_ok = false.
Limpeza de Backups de Áudio	0 4 * * * (diário às 04:00) + boot	Remove pastas de áudio de reuniões com mais de 7 dias ou cuja ata já expirou (audio_expira_em).
Cron de Advertências	0 6 * * * (diário às 06:00)	Marca como "sem_resposta" advertências sem manifestação há > 7 dias e expira advertências vigentes há > 90 dias.
Cron de Lembretes do Termo	0 9 * * * (diário às 09:00)	Envia lembretes em DM para assinatura do Termo Gov.br e alerta o RH sobre prazos vencidos (> 7 dias).

9.3. Catálogo de Comandos Slash e Permissões Exigidas

Comando Slash	Permissão / Escopo	Comportamento e Regras Principais
/declaracao solicitar	Membros Cadastrados	Gera declaração PDF (4h regular / 10h líder), cria protocolo #DECL- e envia por SMTP ao coordenador.
/declaracao status	Membros Cadastrados	Sincroniza a caixa IMAP em tempo real e exibe as últimas 10 solicitações do membro.
/painel-ponto	Administrator (Guild)	Publica o painel interativo fixo de ponto eletrônico com 6 botões de ação.
/ajustar-ponto <membro>	Cargo PONTO_ROLE_RH_ID	Lista últimos 25 pontos fechados do membro para ajuste de horário (máx 24h) com auditoria e DM.
/importar-horas ...	Cargo PONTO_ROLE_RH_ID	Importa ou substitui saldo acumulado (dias, horas, min, seg) sem inflar o ranking semanal.
/painel-advertencias	AUTORES_ADVERTENCIA_IDS	Publica o painel administrativo de advertências, fila e consultas no canal.
/advertir ...	AUTORES_ADVERTENCIA_IDS	Inicia abertura de advertência (Leve, Média, Grave) via parâmetros slash + modal de evidências.
/advertencias [membro]	AUTORES_ADVERTENCIA_IDS	Consulta histórico disciplinar completo do servidor ou de um membro (incluindo fila).
/minhas-advertencias	Qualquer Membro	Exibe histórico privado das próprias advertências (ocultando itens ainda "na_fila").
/reuniao iniciar | finalizar | historico | reprocessar	Membros em Canal de Voz	Grava áudio por falante, transcreve em lotes com Gemini 2.5 Flash, gera ata em PDF e reprocessa backups.